

Architecture MVVM : une responsabilité par couche

ViewModel, StateFlow, flux unidirectionnel
Mini-TP « Compléter la couche manquante »



Trois séances, une question — et sa réponse

3

Séance 3 — le constat

Le système DÉTRUIT et RECRÉE l'Activity à la rotation. Vous l'avez vu au Logcat : le numéro d'instance change.

4

Séance 4 — la limite

remember survit aux recompositions, mais pas à la rotation : votre compteur est retombé à zéro.

6

Aujourd'hui — la réponse

Le ViewModel vit PLUS LONGTEMPS que l'Activity : le système le conserve à travers la recreation. L'état y est en sécurité.

« Ne mettez pas l'état dans l'écran : l'écran est jetable, l'état ne doit pas l'être. »



Les trois couches : qui fait quoi

UI — Compose

AFFICHE l'état • **SIGNALE** les gestes

Ne calcule rien, ne décide rien, ne va chercher aucune donnée.



ViewModel

PORTE l'état • **TRAITE** les événements

Le cerveau de l'écran — et il ne connaît rien d'Android : testable sans téléphone.



Repository (source)

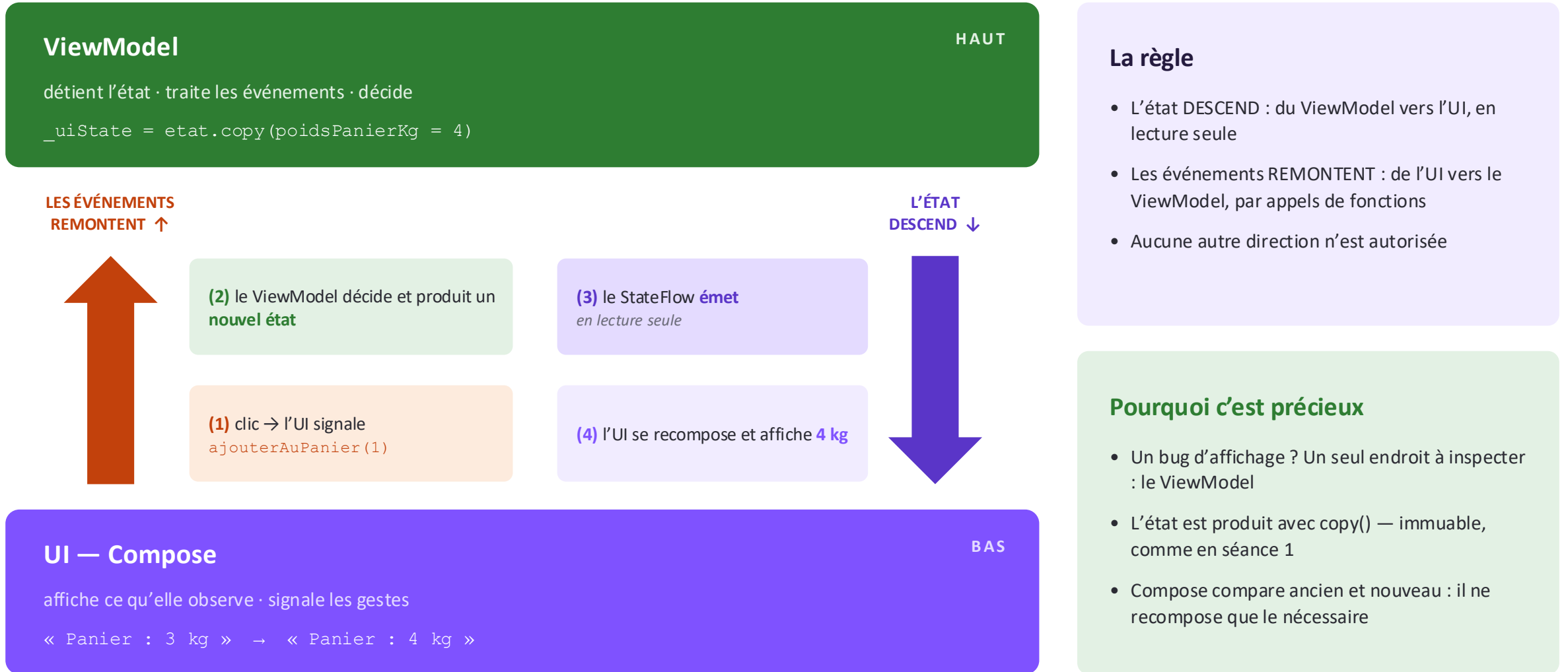
FOURNIT les données

Seul à savoir d'où elles viennent : mémoire aujourd'hui, base Room en séance 7.

Une responsabilité par couche : chacune peut changer — ou être testée — sans casser les autres.



Le flux unidirectionnel : l'état descend, les événements remontent



Le ViewModel est en haut parce qu'il DÉTIENT l'état. Un tour complet par interaction : on revient au départ, mais avec un état différent.



Le ViewModel en code : le duo `_uiState` / `uiState`

ProduitsViewModel.kt — les deux TODO du mini-TP

```
data class EtatUi(  
    val produits: List<Produit> = emptyList(),  
    val poidsPanierKg: Int = 0,  
)  
  
class ProduitsViewModel : ViewModel() {  
  
    // TODO 1 — l'état : le duo  
    private val _uiState = MutableStateFlow(EtatUi(...))  
    val uiState: StateFlow<EtatUi> = _uiState  
  
    // TODO 2 — l'événement  
    fun ajouterAuPanier(poidsKg: Int) {  
        _uiState.update { etat ->  
            etat.copy(  
                poidsPanierKg = etat.poidsPanierKg + poidsKg  
            )  
        }  
    }  
}
```

Côté UI — observer et signaler

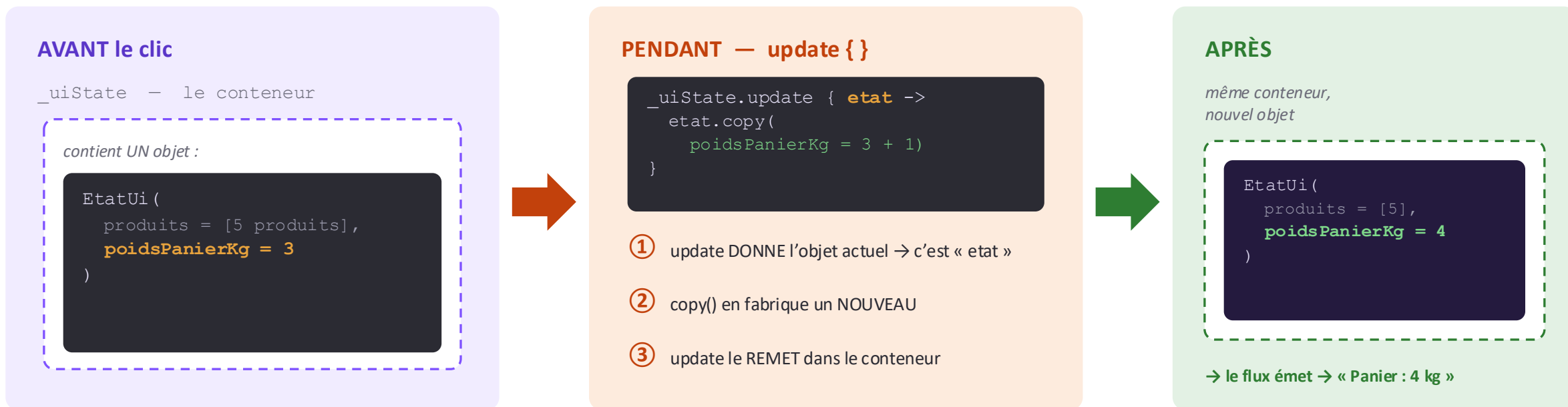
```
// UN ViewModel partagé  
// par les deux écrans :  
val viewModel: ProduitsViewModel  
    = viewModel()  
  
// OBSERVER l'état  
val etat by viewModel.uiState  
    .collectAsState()  
Text("Panier : ${etat.poidsPanierKg} kg")  
  
// SIGNALER un événement  
Button(onClick = {  
    viewModel.ajouterAuPanier(1)  
}) { Text("Ajouter 1 kg") }
```

`_uiState` privé et mutable · `uiState` public et en lecture seule : le flux unidirectionnel est garanti par les TYPES, pas par la discipline.



Un clic, pas à pas : ce que devient l'état

ajouterAuPanier ne MODIFIE rien : elle fabrique un nouvel EtatUi et le met à la place de l'ancien dans le conteneur.



Pourquoi « produits » n'apparaît pas dans copy()

Les champs non cités sont recopiés tels quels. On ne réécrit que ce qui change — c'est tout l'intérêt de copy().

Pourquoi un objet NEUF, et pas une modification

L'ancien n'est jamais touché (les champs sont des val). C'est ce qui permet à Compose de comparer avant/après et de ne redessiner que le texte du panier.

La data class décrit la **FORME** de l'état et fournit `copy()` · la fonction décrit une **TRANSITION** vers l'état suivant.



Mini-TP 6 - « Compléter la couche manquante »

1 Lire et prédire

Lire les deux fichiers ; prédire : que fait « Ajouter 1 kg » AVANT les TODO ? et que devient le panier à la rotation, une fois les TODO faits ?

2 Les deux TODO

TODO 1 : le duo `_uiState / uiState`. TODO 2 : l'événement (update + copy). Vérifier : panier partagé entre les écrans, puis rotation → il survit.

3 Casser le flux

Décommenter la variable « triche » hors circuit et son bouton. Cliquer : l'affichage ne bouge pas. Tourner l'écran : que se passe-t-il ? Expliquer.

4 Voie ouverte — trier une critique

Critique IA de votre ViewModel ; trier chaque remarque : pertinente / non pertinente ici, avec un mot de justification.

Le moment de la séance : vous tournerez l'écran, et le panier sera toujours là. La question ouverte depuis la séance 3 se referme.

